Table des matières

Exercices corrigés 05 — Le code d'un pipeline robuste 1
Questions courtes 1
Questions pratiques 3
Diagnostics 3
Questions à développer 4
Mises en situation 4
Corrigé 5

Exercices corrigés 05 — Le code d'un pipeline robuste

Comment travailler ces exercices. Répondez seul, puis vérifiez avec le corrigé. Les exercices marqués [labo] portent sur le pipeline que vous venez d'écrire ; les autres vérifient que le cours est clair dans votre tête. Tout vient de la Théorie 05 et du Labo 05.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Ce qui se configure. Donnez deux valeurs du pipeline de Boréal qui appartiennent à `config.toml`, et deux qui n'y ont pas leur place bien qu'elles soient écrites dans le code.
2. Le mot de passe et le fichier versionné. Pourquoi un mot de passe ne se range-t-il pas dans `config.toml`, même en dernière ligne, même commenté ?
3. Un modèle sans valeur. Qu'est-ce que `.env.exemple` apporte, puisqu'il ne contient aucune valeur utilisable ?
4. La surcharge. Quelle variable d'environnement donnerait 5434 au port de la base de l'ERP, sans toucher au fichier de configuration ?
5. D'où vient un secret. Le code appelle `secret("ERP_MOT_DE_PASSE")`. Nommez trois endroits d'où cette valeur peut venir, selon l'environnement, sans que la ligne de code change.
6. Deux traces. Quelle est la différence de nature entre le fichier `pipeline_2026-09-15.log` et la table `pipeline.executions` ?

© 2026 iAlexMG. Tous droits réservés.

7. Le bon niveau. À quel niveau de journal inscrire : le début d'une étape, un essai raté suivi d'une reprise réussie, une étape qui s'arrête, le détail de chaque page d'API ?
8. La ligne écrite d'avance. La ligne de `pipeline.executions` s'écrit avant le travail, à l'état « en cours ». Qu'est-ce qu'on perdrait à l'écrire seulement à la fin ?
9. Passager ou définitif. Classez ces réponses : `429`, `401`, `503`, `404`, un délai dépassé, une réponse JSON mal formée.
10. Compter les attentes. Quatre essais et une attente de 2 secondes qui double : combien de temps d'attente en tout avant l'abandon, et combien d'attentes ?
11. Pourquoi doubler. Pourquoi l'attente double-t-elle au lieu de rester à 2 secondes ?
12. Ce que la reprise suppose. Pourquoi les reprises du Cours 05 supposent-elles l'idempotence du Cours 03 ?
13. L'horloge donnée de l'extérieur. La fonction `avec_reprises` reçoit un argument `dormir`. À quoi sert-il, et qui s'en sert ?
14. Un test plus exigeant. Le test de la reprise vérifie `attentes == [2.0, 4.0, 8.0]`. En quoi est-ce plus fort que de vérifier que l'abandon a bien eu lieu ?
15. Ce qu'on ne teste pas. Nommez deux chemins du pipeline que les trois tests ne couvrent pas, et dites comment ils se vérifient.
16. Un projet jetable. Pourquoi les tests posent-ils `PIPELINE_PROJET` sur un dossier temporaire ?
17. Deux commandes, deux résultats. `pytest` échoue sur `ModuleNotFoundError: No module named 'pipeline'`, mais `python -m pytest` passe. Pourquoi, et que corrige `pytest.ini` ?
18. Le fichier qui rend un paquet exécutable. Quel fichier faut-il dans un dossier pour que `python -m <dossier>` l'exécute ?
19. Le code de sortie. Le pipeline rend 0 ou 1. Qui lit cette valeur, et que se passerait-il s'il rendait toujours 0 ?

1. Hors du chemin hors ligne. Pourquoi `erp` et `meteo` ne font-elles pas partie de ce que lance `tout` ?
2. L'ordre des étapes. `python -m pipeline raw bronze` est accepté par la porte d'entrée. Est-ce une bonne idée ?
3. Un import déplacé. Dans `erp.py`, `import pyodbc` est écrit à l'intérieur de la fonction. Quelle serait la conséquence de le remonter en tête du module ?
4. Dedans, dehors. Citez trois choses que le `Dockerfile` met dans l'image, et trois qu'il laisse dehors.
5. Une version épinglée. Pourquoi `duckdb==1.5.5` plutôt que `duckdb` tout court ?
6. Deux garde-fous. `.gitignore` et `.dockerignore` nomment tous deux `.env`. Contre quoi chacun protège-t-il ?
7. Le fichier du lendemain. Le conteneur écrit `plateforme/journaux/pipeline_2026-09-16.log` alors qu'il est 20 h 30 le 15 chez vous. Pourquoi ?

© 2026 iAlexMG. Tous droits réservés.

8. Le lignage qui traverse. Le lignage de la `RAW` garde `sources/erp/extraits/2023-01/VBAK.csv`. Qu'arriverait-il, au premier lancement dans le conteneur, s'il gardait `C:\...\sources\erp\extraits\2023-01\VBAK.csv` ?
9. La dernière instruction. Le module `bronze.py` lit le résultat de la dernière instruction des scripts SQL du Cours 02. Que se passe-t-il si quelqu'un ajoute une requête après le contrôle ?

Questions pratiques

1. [labo] Le mode bavard. Lancez le pipeline en mode bavard sur la seule étape du corpus, et dites quelles lignes apparaissent qui ne s'affichaient pas.
2. [labo] Le profil des étapes. Écrivez la requête qui donne, pour chaque étape, le nombre de passages, la durée moyenne et la durée la plus longue, de la plus lente à la plus rapide.
3. [labo] Les exécutions tuées. Écrivez la requête qui trouve les exécutions restées à l'état « en cours », puis fabriquez-en une pour vérifier qu'elle les voit.
4. [labo] Du tableau au journal. Une étape a échoué hier soir. Écrivez la requête qui rend son identifiant d'exécution et son message, puis dites comment retrouver le détail dans les fichiers de journal.
5. [labo] Un contrôle qui tombe. Faites perdre des lignes à la conversion du bronze en ajoutant un filtre au `COPY` de `sql/bronze_historien.sql`, et observez ce que le pipeline écrit dans les deux traces. Remettez le fichier en état ensuite.
6. [labo] Une station pour un seul lancement. Sans toucher à `config.toml`, faites lire au pipeline la station de météo `7060401`, pour un lancement seulement.
7. [labo] Un quatrième test. Écrivez le test qui vérifie que la surcharge par l'environnement convertit bien `PIPELINE_ERP_PORT` en entier, et non en chaîne de caractères.
8. [labo] Le coût de la frontière. Mesurez l'écart de durée de l'étape `raw` entre un lancement sous Windows et un lancement dans le conteneur, sur un projet déjà chargé, et expliquez-le.

Diagnostics

1. Un collègue lance `python -m pipeline tout` et reçoit `FileNotFoundError: ...\01_de\pipeline\config.toml est introuvable`. Le fichier existe pourtant. Que s'est-il passé ?
2. Le pipeline tourne toutes les nuits par un ordonnanceur. Depuis trois semaines, `pipeline.executions` montre `raw` en « réussi » avec 0 ligne chaque nuit, et personne ne s'en est inquiété. Le fournisseur a cessé de livrer les extraits il y a trois semaines. Qu'est-ce qui manque au pipeline ?
3. Une étape qui appelle une API répond en 40 minutes au lieu de 2, sans erreur. Les journaux montrent des lignes `WARNING ... essai 1 raté (HTTP 401), nouvel essai dans 2 s`. Qu'est-ce qui a été mal programmé ?

© 2026 iAlexMG. Tous droits réservés.

4. Le conteneur recharge les 1 572 fichiers à chaque lancement, alors que Windows dit « à jour ». Donnez deux causes possibles et la façon de trancher entre elles.

Questions à développer

1. Une étape de plus. Votre équipe veut ajouter une étape `garanties`, qui anonymise les données personnelles du portail avant de les livrer. Décrivez ce que vous écrivez, où, et ce que vous ajoutez aux tests, sans écrire le code.
2. La configuration par l'environnement seul. Un architecte propose de remplacer `config.toml` par des variables d'environnement seules, « comme le veut la méthode des douze facteurs », un ensemble de règles pour les applications déployées en continu, dont l'une veut que la configuration vienne de l'environnement et jamais du code. Répondez-lui : ce qu'il gagne, ce qu'il perd, et ce que vous proposez.
3. Les journaux au bout de deux ans. Le pipeline écrit ses journaux dans `plateforme/journaux/`, un fichier par jour, sans limite. Décrivez ce qui arrive au bout de deux ans, et deux façons de traiter la question.

Mises en situation

1. Chez Boréal, un ingénieur a validé un `.env` avec le mot de passe de la base de l'ERP il y a six mois, dans le dépôt interne. Il vient de s'en apercevoir et retire le fichier au commit du jour. Que lui répondez-vous, et dans quel ordre agissez-vous ?
2. L'équipe de l'usine veut lancer le pipeline elle-même, sans installer Python. On vous demande si l'image du Cours 05 suffit. Que vérifiez-vous avant de dire oui, et quelles limites annoncez-vous ?
3. Un pipeline voisin, écrit par une autre équipe, réessaie indéfiniment toute erreur, « pour ne jamais perdre une nuit ». Il n'a jamais échoué en un an. Que soupçonnez-vous, et comment le vérifiez-vous ?

© 2026 iAlexMG. Tous droits réservés.

Corrigé

La numérotation est celle des énoncés. Les chiffres sont ceux de la génération par défaut de Boréal, le pipeline du Labo 05 en place. Les requêtes se tapent dans l'interface web, une par cellule, après `ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS entrepot (READ_ONLY);`, les tables nommées en entier ; l'entrepôt se détache (`DETACH entrepot;`) avant toute commande du pipeline. Les commandes se tapent dans la fenêtre 1 du labo, depuis `Projet_Boreal`, après `set PYTHONPATH=01_de`.

Questions courtes

1. Dans `config.toml` : le port de la base de l'ERP (5433 chez vous, autre chose ailleurs), le chemin de l'entrepôt, la station de météo, le nombre d'essais, la taille des paquets ODBC. Pas dans `config.toml` : la liste `LIVRAISONS` des 22 fichiers de Boréal et la forme de chacun, les requêtes de `LECTEURS`. Ce ne sont pas des réglages, c'est ce que le pipeline sait de ses sources ; les sortir disperserait la lecture sans rien rendre configurable d'utile.

Théorie : `Theorie_05_DE.md`, section 2.

2. Parce que `config.toml` est versionné, et que Git n'oublie rien. Un mot de passe validé une fois reste lisible dans l'historique, même retiré au commit suivant ; l'en retirer vraiment demande de réécrire l'historique du dépôt. Un commentaire n'y change rien : le fichier est lu par tout le monde, et le commentaire aussi.

Théorie : `Theorie_05_DE.md`, section 3.

3. Il dit quelles clés existent, sans donner de valeur. Celui qui arrive sur le projet sait qu'il lui faut `ERP_MOT_DE_PASSE`, sans avoir à lire le code ni à demander. C'est de la documentation exécutable : `copy .env.exemple .env`, et il ne reste qu'à remplir.

Théorie : `Theorie_05_DE.md`, section 3.

4. `PIPELINE_ERP_PORT=5434`. Le nom se forme de `PIPELINE_`, de la section en majuscules, et de la clé en majuscules. La valeur arrive en texte, et `_surcharger` la reconvertit dans le type de la valeur du fichier : ici, un entier.

Théorie : `Theorie_05_DE.md`, section 2.

5. Trois endroits, du poste au serveur : le fichier `01_de/.env` ; une variable d'environnement du terminal ou du conteneur, qui gagne sur le fichier ; et, en entreprise, un gestionnaire de secrets interrogé au démarrage, qui remplirait l'environnement ou répondrait à `secret()`. La ligne de code ne change dans aucun des trois cas : elle ne connaît que le nom.

Théorie : `Theorie_05_DE.md`, section 3.

6. Le fichier est une suite de lignes de texte datées, qu'on lit dans l'ordre quand on cherche pourquoi quelque chose s'est passé. La table est un état interrogeable, qu'on questionne en SQL pour comparer : combien de fois, depuis quand, avec quelle durée, avec quel écart. L'un raconte, l'autre compte, et aucun ne remplace l'autre.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_05_DE.md`, section 4.

7. Le début d'une étape : `INFO`. Un essai raté suivi d'une reprise réussie : `WARNING`, parce que rien n'est perdu mais que ce n'est pas normal. Une étape qui s'arrête : `ERROR`. Le détail de chaque page d'API : `DEBUG`, qu'on n'allume que pendant qu'on cherche.

Théorie : `Theorie_05_DE.md`, section 4.

8. On perdrait la trace des exécutions qui n'ont jamais fini. Un processus tué en plein travail, une machine qui redémarre, un conteneur arrêté : la ligne écrite seulement à la fin ne s'écrirait jamais, et la table dirait que rien ne s'est passé cette nuit-là. Avec l'écriture au début, il reste une ligne « en cours » qui se voit, et la question se pose.

Théorie : `Theorie_05_DE.md`, section 4.

9. Passagères : `429` (trop de requêtes), `503` (service indisponible), le délai dépassé. Définitives : `401` (pas authentifié), `404` (n'existe pas). La réponse JSON mal formée est le cas ambigu : elle est définitive si le service répond toujours ainsi, passagère si une réponse a été tronquée. Le pipeline de Boréal ne la réessaie pas, parce que `passagere` ne reconnaît que les erreurs d'`urllib` ; c'est un choix, et il se discute.

Théorie : `Theorie_05_DE.md`, section 5.

10. Trois attentes, de 2, 4 et 8 secondes, soit 14 secondes en tout. Quatre essais donnent trois intervalles, pas quatre : après le dernier essai, on abandonne au lieu d'attendre.

Théorie : `Theorie_05_DE.md`, section 5.

11. Pour la source autant que pour nous. Un service déjà en difficulté n'a pas besoin de clients qui le rappellent chaque seconde : en lui laissant de l'air, on augmente sa chance de se rétablir, donc la nôtre d'obtenir une réponse. Une attente fixe et courte, multipliée par les clients, achève un service qui vacille.

Théorie : `Theorie_05_DE.md`, section 5.

12. Parce qu'une reprise refait l'appel, et qu'on ne peut refaire sans dégât que ce qui est rejouable. Une lecture l'est toujours ; une écriture qui ajoute ne l'est pas. Le chargeur du Cours 03 peut se relancer parce que son journal des chargements l'empêche de doubler quoi que ce soit : sans cette propriété, chaque reprise créerait des doublons silencieux.

Théorie : `Theorie_05_DE.md`, section 5.

13. `dormir` est la façon d'attendre, donnée de l'extérieur. En production, c'est `time.sleep`. Dans le test, c'est la méthode `append` d'une liste, qui note le délai au lieu de le subir : le test vérifie les attentes sans jamais dormir, et tourne en quelques millisecondes.

Théorie : `Theorie_05_DE.md`, section 6.

14. Vérifier l'abandon prouve seulement que la boucle finit. Vérifier `[2.0, 4.0, 8.0]` prouve la politique elle-même : le nombre d'essais, la valeur de départ et le doublement. Un code qui attendrait 2 secondes trois fois de suite passerait le premier contrôle et échouerait au second.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_05_DE.md`, section 6.

15. L'extraction ODBC, qui demande le conteneur de la base et le pilote psqlODBC ; l'appel à la vraie API d'ECCC, qui demande le réseau. Ni l'un ni l'autre ne se teste automatiquement sans rendre les tests lents et dépendants d'un service extérieur. Ils se vérifient en les exécutant, par les contrôles des Labos 02 et 03 : 0 écart sur les neuf tables de l'ERP, et les jours reçus égaux aux jours annoncés par l'API.

Théorie : `Theorie_05_DE.md`, section 6.

16. Pour que les tests n'écrivent rien dans le projet. Sans cela, ils créeraient des tables dans votre entrepôt, y laisseraient des lignes de journal, et échoueraient selon l'état où le dernier labo l'a laissé. Un test doit partir d'un état qu'il connaît, et ne rien laisser derrière.

Théorie : `Theorie_05_DE.md`, section 6.

17. `python -m pytest` ajoute le dossier courant aux chemins d'importation, `pytest` non. Le paquet `pipeline` vit dans `01_de`, donc il n'est trouvé que dans le premier cas. `pytest.ini` règle la question une fois pour toutes avec `pythonpath = .`, et les deux commandes marchent.

Théorie : `Theorie_05_DE.md`, section 6.

18. Un fichier `__main__.py`. C'est lui que `python -m <paquet>` exécute. Le `__init__.py`, lui, fait du dossier un paquet importable, ce qui est une autre question.

Théorie : `Theorie_05_DE.md`, section 7.

19. L'ordonnanceur, Airflow au Cours 06, et tout ce qui lance le pipeline sans regarder l'écran. S'il rendait toujours 0, une étape en échec serait comptée comme réussie : aucune alerte, aucune nouvelle tentative, et la tâche suivante partirait sur des données incomplètes. Un journal qui dit « ERREUR » ne remplace pas un code de sortie, parce que personne ne lit les journaux d'un travail réussi.

Théorie : `Theorie_05_DE.md`, section 7.

20. Parce qu'elles supposent autre chose que le projet : `erp` demande le conteneur PostgreSQL et le pilote psqlODBC, `meteo` demande un accès au réseau. Un pipeline qu'on doit pouvoir rejouer hors ligne, sur n'importe quel poste, ne les suppose pas. Elles restent appelables par leur nom quand leurs conditions sont réunies.

Théorie : `Theorie_05_DE.md`, section 7.

21. Non. La porte d'entrée lance les étapes dans l'ordre donné, sans rien vérifier, et `raw` avant `bronze` n'a pas de sens : la `RAW` lit les fichiers livrés, le bronze range l'historien et la télémétrie, et le Cours 06 montrera que c'est le graphe des dépendances qui doit porter cet ordre. Ici, la commande obéit ; c'est à vous de savoir ce que vous demandez.

Théorie : `Theorie_05_DE.md`, section 7.

© 2026 iAlexMG. Tous droits réservés.

22. Le pipeline entier refuserait de démarrer sur un poste sans `pyodbc`, même pour charger la `RAW`, parce que `__main__.py` importe tous les modules d'étape au chargement. L'import dans la fonction fait payer la dépendance à qui s'en sert seulement. C'est la condition pour que le chemin rejoué par le kit de départ d'Analytics Engineering n'ait besoin que de DuckDB et de la bibliothèque standard.

Théorie : `Theorie_05_DE.md`, section 7.

23. Dans l'image : Python 3.11, DuckDB dans une version fixée, le paquet `pipeline` et les scripts SQL. Dehors : les données, qui arrivent par un montage ; les secrets, qui arrivent par l'environnement ; les tests, qui appartiennent à l'atelier et non au pipeline livré.

Théorie : `Theorie_05_DE.md`, section 8.

24. Parce que sans le numéro, l'image reconstruite dans six mois prendra la version du jour. Un moteur SQL qui change de version peut changer un type deviné, un ordre de tri, un comportement de lecture. Le Cours 01 a montré ce que la détection de types fait aux extraits de l'ERP : ce n'est pas le genre de chose qu'on veut voir bouger sans l'avoir décidé.

Théorie : `Theorie_05_DE.md`, section 8.

25. `.gitignore` empêche le `.env` d'entrer dans l'historique du dépôt, donc de partir chez un collègue ou sur une plateforme d'hébergement. `.dockerignore` l'empêche d'entrer dans l'image, donc de partir dans un registre d'images, où il serait lisible par quiconque tire l'image. Deux sorties différentes, deux garde-fous.

Théorie : `Theorie_05_DE.md`, section 8.

26. Parce qu'un conteneur Linux tourne en temps universel coordonné, UTC, tant qu'on ne lui dit pas autre chose. À 20 h 30 heure de l'Est, il est déjà 0 h 30 le lendemain en UTC, et le nom du fichier suit. C'est une bonne raison de dater les journaux en UTC partout plutôt que de deviner : la table `pipeline.executions`, elle, le fait déjà, ses colonnes étant en `TIMESTAMPTZ`.

Théorie : `Theorie_05_DE.md`, section 8.

27. Le journal des chargements ne reconnaîtrait aucun fichier, parce que le conteneur cherche `sources/...` et trouverait `C:\...` dans la table. Les 1 572 fichiers seraient rechargés, et la `RAW` doublerait : chaque ligne existerait deux fois, une par forme de chemin. Le lignage relatif en barres obliques n'est pas une coquetterie, c'est ce qui rend la `RAW` lisible des deux côtés de la frontière.

Théorie : `Theorie_05_DE.md`, section 8.

28. `convertir` lit le résultat de la dernière instruction : ce serait cette nouvelle requête, pas le contrôle. Si elle rend deux colonnes, le pipeline comparerait deux nombres qui n'ont rien à voir ; sinon, il lèverait `EcartDeConversion` avec le message « ne finit pas par un contrôle à deux colonnes ». C'est une fragilité assumée, et le cours la nomme : le contrôle est la dernière instruction, par convention.

Théorie : `Theorie_05_DE.md`, section 6.

Questions pratiques

© 2026 iAlexMG. Tous droits réservés.

1. Dans la fenêtre 1 :

```
python -m pipeline corpus --bavard
```

Deux lignes `DEBUG` apparaissent, qui ne s'affichaient pas : `journal ouvert dans ...\plateforme\journaux\pipeline_AAAA-MM-JJ.log`, écrite par `demarrer`, et `entrepôt ouvert : ...\boreal.duckdb`, écrite par `ouvrir`. Le reste est identique. Ces deux lignes disent où le pipeline écrit et quel fichier il a ouvert : c'est exactement ce qu'on veut savoir quand on cherche pourquoi il ne travaille pas là où on croit.

Théorie : `Theorie_05_DE.md`, section 4.

2. Dans l'interface web, l'entrepôt attaché :

```
-- Le profil de chaque étape, la plus lente en premier.
-- Attendu : raw, bronze, erp, puis corpus ; erp autour de 4 s, le temps que le port refuse la connexion.
SELECT etape,
       COUNT(*)               AS passages,
       ROUND(AVG(duree_s), 1) AS moyenne_s,
       ROUND(MAX(duree_s), 1) AS pire_s
FROM entrepot.pipeline.executions
WHERE fin IS NOT NULL
GROUP BY etape
ORDER BY moyenne_s DESC;
```

Chez l'auteur, après le labo : `raw` 2 passages, 9,5 s de moyenne et 12,8 s au pire (le premier passage complet, puis le conteneur) ; `bronze` 6,5 s ; `erp` 4,1 s ; `corpus` 0,2 s. Le `WHERE fin IS NOT NULL` n'est pas décoratif : une exécution restée « en cours » n'a pas de durée, et sa moyenne fausserait la lecture.

Théorie : `Theorie_05_DE.md`, section 4.

3. La requête :

```
-- Les exécutions qui n'ont jamais fini : processus tué, machine redémarrée, conteneur arrêté.
-- Attendu : aucune ligne sur un projet sain.
SELECT execution_id, etape, debut
FROM entrepot.pipeline.executions
WHERE etat = 'en cours'
ORDER BY debut;
```

Pour en fabriquer une, détachez l'entrepôt, lancez `python -m pipeline bronze` dans la fenêtre 1 et interrompez-le par Ctrl+C pendant qu'il travaille, puis rattachez l'entrepôt : la requête rend alors une ligne. La ligne reste à « en cours » parce que `etape` n'attrape que les exceptions ordinaires, et qu'une interruption n'en est pas une. C'est le comportement voulu : une exécution tuée doit se voir, pas se maquiller en échec propre. Relancez l'étape ensuite, le bronze étant resté à moitié écrit.

Théorie : `Theorie_05_DE.md`, section 4.

© 2026 iAlexMG. Tous droits réservés.

4. La requête :

```
-- Le dernier échec, avec de quoi le retrouver dans les journaux de texte.
SELECT execution_id, etape, debut, message
FROM entrepot.pipeline.executions
WHERE etat = 'échoué'
ORDER BY debut DESC
LIMIT 1;
```

L'identifiant d'exécution, huit caractères, est écrit dans chaque ligne du fichier de journal qui concerne cette étape. La date de `debut` donne le fichier, `plateforme/journaux/pipeline_AAAA-MM-JJ.log`, et une recherche de l'identifiant dans ce fichier donne le déroulement complet, avertissements compris. La table dit qu'il y a eu un échec et lequel ; le fichier dit ce qui s'est passé avant.

Théorie : `Theorie_05_DE.md`, section 4.

5. Dans `sql/bronze_historien.sql`, ajoutez un filtre au `COPY`, juste avant la parenthèse qui le ferme :

```
    )
    WHERE Quality = 192
) TO 'plateforme/lac/bronze/usine/historien'
```

Puis, l'entrepôt détaché, `python -m pipeline bronze` dans la fenêtre 1. Le pipeline s'arrête sur :

```
ERROR   pipeline : étape bronze (...) : échec, historien : 275 436 lignes dans la source, 275 080
dans le bronze. La conversion a perdu ou ajouté des lignes.
```

Les deux traces se complètent : le fichier de journal porte la ligne ci-dessus, et `pipeline.executions` garde une ligne `bronze` en « échoué » avec ce message. Le code de sortie vaut 1. Remettez le fichier en état et relancez : le contrôle retrouve 275 436 des deux côtés.

Notez ce que l'exercice démontre. Le filtre ne provoque aucune erreur : DuckDB fait exactement ce qu'on lui demande, et écrit un bronze amputé des 356 lignes de qualité douteuse ou mauvaise. Sans le contrôle, le pipeline aurait dit « terminé » et personne n'aurait rien vu.

Théorie : `Theorie_05_DE.md`, section 1.

6. Pour un seul lancement, dans la fenêtre 1 :

```
set PIPELINE_METEO_STATION=7060401
python -m pipeline meteo
set PIPELINE_METEO_STATION=
```

La station lue devient `7060401`, et rien d'autre ne change : `page` vaut toujours 50, les dates restent celles du fichier. La dernière commande vide la variable ; sans elle, tous les lancements de ce terminal viseraient la mauvaise station, et rien ne le signalerait.

Théorie : `Theorie_05_DE.md`, section 2.

© 2026 iAlexMG. Tous droits réservés.

7. Dans `01_de\tests\test_config.py` (`notepad 01_de\tests\test_config.py`, dans la fenêtre 1) :

```python
# -*- coding: utf-8 -*-
"""La surcharge par l'environnement garde le type de la valeur du fichier."""
import pytest

from pipeline.config import config


@pytest.fixture
def config_neuve(monkeypatch):
    config.cache_clear()
    yield monkeypatch
    config.cache_clear()


def test_la_surcharge_garde_le_type(config_neuve):
    """PIPELINE_ERP_PORT arrive en texte, mais la configuration rend un entier."""
    assert config()["erp"]["port"] == 5433

    config_neuve.setenv("PIPELINE_ERP_PORT", "5434")
    config.cache_clear()
    port = config()["erp"]["port"]
    assert port == 5434
    assert isinstance(port, int), "un port en texte casserait la chaîne de connexion ODBC"


def test_la_surcharge_d_une_liste_coupe_sur_les_virgules(config_neuve):
    """Une liste du fichier reste une liste, coupée sur les virgules et débarrassée des espaces."""
    config_neuve.setenv("PIPELINE_ERP_TABLES", "vbak, vbap")
    config.cache_clear()
    assert config()["erp"]["tables"] == ["vbak", "vbap"]
```

`pytest 01_de` rend alors `5 passed`, en 0,28 s. Les deux `cache_clear` encadrent le test parce que `config()` garde son résultat en mémoire : sans eux, la valeur lue avant la pose de la variable resterait en place, et le test passerait ou échouerait selon l'ordre des tests.

Théorie : `Theorie_05_DE.md`, section 6.

8. Sur un projet déjà chargé, l'étape `raw` ne fait qu'une chose : calculer les empreintes SHA-256 des 1 572 fichiers pour les comparer au journal. Sous Windows, 0,4 s, une fois les fichiers en cache ; dans le conteneur, sur le même projet monté, 6 à 7 s, soit une quinzaine de fois plus. La différence n'est pas le calcul, qui est le même, mais la lecture : les fichiers de Windows vus depuis un conteneur Linux passent par une couche de traduction, et 1 572

© 2026 iAlexMG. Tous droits réservés.

ouvertures de fichiers la paient chacune. C'est le coût annoncé par la décision D59 du parcours, et il explique pourquoi le Cours 06 mesurera la question avant de faire tourner Airflow sur ces fichiers.

Théorie : `Theorie_05_DE.md`, section 8.

Diagnostics

1. Le fichier a été créé au mauvais endroit, dans `01_de/pipeline/` au lieu de `01_de/`. Le message le dit d'ailleurs en entier : il donne le chemin cherché, et ce chemin se termine par `pipeline\config.toml`. `config.py` calcule `DEPOT` comme le dossier qui porte le paquet, donc `01_de` ; le fichier doit être à côté du dossier `pipeline`, pas dedans.

Théorie : `Theorie_05_DE.md`, section 2.

2. Il manque un contrôle sur ce qui n'arrive pas. Le pipeline est idempotent et fait exactement ce qu'on lui demande : aucun fichier nouveau, donc aucune ligne chargée, donc « réussi ». C'est le type de panne le plus coûteux, parce qu'elle ne produit aucune erreur. Ce qu'il faudrait : une attente déclarée, du genre « un extrait mensuel arrive chaque mois », et un contrôle qui échoue quand l'attente n'est pas tenue. C'est exactement l'objet du Cours 11, les contrats de données ; en attendant, la requête de la question pratique 2, lue chaque semaine, aurait montré `raw` à 0 ligne.

Théorie : `Theorie_05_DE.md`, section 4.

3. Un `401` est traité comme une erreur passagère. La fonction `passagere` a été écrite pour réessayer tout `HTTPError`, ou pour tester le code d'erreur à l'envers. Le pipeline réessaie donc quatre fois, avec ses attentes de 2, 4 et 8 secondes, une erreur qui ne passera jamais : le jeton est expiré ou le compte n'a pas le droit. Il faut lire la condition de `passagere` : seuls `429` et les codes à partir de 500 méritent un second essai.

Théorie : `Theorie_05_DE.md`, section 5.

4. Deux causes possibles. Soit le lignage a été écrit en chemins absolus ou à barres inversées, et le conteneur ne reconnaît rien de ce que Windows a chargé. Soit le conteneur ne voit pas le même entrepôt, parce que le montage désigne un autre dossier que `Projet_Boreal` : il repart alors d'un journal des chargements vide.

Pour trancher, une requête suffit, lancée côté Windows, dans l'interface web :

```
-- La forme des chemins du journal des chargements.
-- Attendu : sources/erp/extraits/2023-01/VBAK.csv, relatif, en barres obliques.
SELECT fichier FROM entrepot.pipeline.chargements LIMIT 3;
```

Si les chemins sont relatifs et en barres obliques, la cause est le montage : vérifiez ce que le conteneur voit, par `docker run --rm -v ...:/travail --entrypoint sh <image> -c "ls /travail"`, qui doit montrer `sources`, `plateforme` et `01_de`.

Théorie : `Theorie_05_DE.md`, section 8.

Questions à développer

© 2026 iAlexMG. Tous droits réservés.

1. Un module `pipeline/garanties.py`, avec une fonction `executer(con)` qui rend le nombre de lignes écrites, et une entrée dans le dictionnaire `ETAPES` de `__main__.py`. La règle d'anonymisation elle-même est du domaine : elle vit dans ce module, pas dans `config.toml` ; ce qui va dans la configuration, c'est ce qui change d'un environnement à l'autre, par exemple le sel du hachage, qui est d'ailleurs un secret et va donc dans le `.env`.

Une décision se pose avant d'écrire : la `RAW` garde ce qui est livré, tel quel, et le Cours 03 a posé que les données personnelles y entrent. L'étape anonymise donc au-delà de la `RAW`, vers une table du schéma `argent`, et la question de savoir combien de temps la `RAW` garde les données personnelles est une question de gouvernance, pas de code.

Côté tests : un test qui vérifie que la même entrée donne toujours la même sortie anonymisée, sans quoi les jointures se briseraient ; un test qui vérifie qu'aucune colonne nommée dans la liste des données personnelles ne survit dans la table livrée ; un test qui vérifie que le compte des lignes est conservé, l'anonymisation ne devant ni perdre ni dédoubler.

Théorie : `Theorie_05_DE.md`, sections 2 et 6.

2. Il gagne l'uniformité : une seule façon de configurer, valable du poste au conteneur, et rien à lire sur le disque. Il perd trois choses. La structure d'abord : une liste de neuf tables ou une section entière deviennent des chaînes à découper, et les erreurs se découvrent à l'exécution. La mémoire ensuite : un fichier versionné garde l'historique des valeurs et de leurs raisons, un environnement ne garde rien. La lisibilité enfin : `config.toml` se lit d'un coup d'œil, et ses commentaires expliquent pourquoi le port est 5433.

Ce que le pipeline fait déjà est la réponse : le fichier donne les valeurs par défaut, et l'environnement les surcharge, valeur par valeur. La méthode des douze facteurs demande que la configuration ne soit pas figée dans le code ; elle est respectée dès lors que tout se surcharge depuis l'extérieur. Il n'y a pas à choisir entre les deux.

Théorie : `Theorie_05_DE.md`, section 2.

3. Au bout de deux ans, environ 730 fichiers, dont la taille dépend du niveau de journal ; ce n'est pas un problème d'espace, c'est un problème de recherche. Personne ne lit 730 fichiers, et un dossier qui grossit sans fin finit par être ignoré, ce qui revient à ne plus journaliser du tout.

Deux traitements. Le premier, la rotation : `logging.handlers.TimedRotatingFileHandler` garde les trente derniers jours et efface le reste, la trace longue durée restant dans `pipeline.executions`, qui est compacte et interrogeable. Le second, la centralisation : les journaux partent vers un service qui les indexe et les cherche, ce qui devient utile dès qu'il y a plusieurs machines. Pour Boréal, la rotation suffit, et le choix se dit dans le cours plutôt que se subir.

Théorie : `Theorie_05_DE.md`, section 4.

Mises en situation

1. L'ordre compte, et le retrait du fichier est la dernière étape, pas la première.

D'abord, considérer le mot de passe comme compromis et le changer, dans la base et dans le gestionnaire de secrets. C'est le seul geste qui répare vraiment : tant que l'ancien fonctionne, il est lisible par quiconque a cloné le dépôt depuis six mois, et il l'est même après réécriture de l'historique, dans les copies locales et les caches de la plateforme d'hébergement.

© 2026 iAlexMG. Tous droits réservés.

Ensuite, regarder ce que ce compte pouvait faire, et pendant combien de temps. Six mois de dépôt interne, c'est une liste de personnes qui se dresse.

Ensuite seulement, nettoyer : retirer le fichier, vérifier qu'il est bien dans `.gitignore`, et décider si l'historique se réécrit, ce qui n'est pas anodin sur un dépôt partagé.

Enfin, éviter la prochaine fois : un contrôle avant chaque commit qui refuse un fichier `.env`, et le `.env.exemple` comme seule forme versionnée.

Ce qu'on ne dit pas : que ce n'est pas grave parce que le dépôt est interne. La majorité des fuites de secrets viennent de dépôts qu'on croyait fermés.

Théorie : `Theorie_05_DE.md`, section 3.

2. L'image suffit pour lancer le pipeline, elle ne suffit pas pour le projet.

Ce qu'on vérifie avant de dire oui : que le poste de l'usine a un moteur de conteneurs, ce qui n'est pas rien à l'usine ; que le dossier `Projet_Boreal` lui est accessible, avec le droit d'écrire, parce que le pipeline écrit dans l'entrepôt et le lac ; qu'une seule personne à la fois lance le pipeline, la règle d'un seul processus sur l'entrepôt valant aussi entre conteneurs.

Les limites à annoncer : les étapes `erp` et `meteo` ne tourneront pas sans le conteneur de la base ni le réseau ; l'accès aux fichiers à travers la frontière est lent, une quinzaine de fois plus sur l'étape `raw` ; et l'image fige une version du code, donc il faudra la reconstruire à chaque changement. Si l'équipe de l'usine doit surtout lire des résultats plutôt que lancer des traitements, la vraie réponse est ailleurs : un accès au tableau de bord, que le Cours 12 mettra en place.

Théorie : `Theorie_05_DE.md`, section 8.

3. Un pipeline qui n'a jamais échoué en un an n'est pas un pipeline robuste, c'est un pipeline qui ne dit rien. Les reprises infinies transforment une panne visible en attente silencieuse : la tâche reste en cours, le code de sortie ne vient jamais, et l'ordonnanceur croit que le travail avance.

Ce qu'on vérifie. D'abord les durées : une distribution des temps d'exécution montrera des nuits à quarante minutes au lieu de deux, signe de reprises en série. Ensuite les comptes livrés, jour par jour : des jours à zéro ligne qui passent pour des succès sont la même panne, vue de l'autre côté. Enfin les journaux, à la recherche d'avertissements de reprise, et de reprises sur des erreurs qui ne sont pas passagères.

La correction tient en trois points : borner les essais, distinguer les erreurs passagères des définitives, et faire échouer bruyamment ce qui ne passe pas. Un pipeline doit échouer quand il échoue ; c'est la seule façon qu'on le répare.

Théorie : `Theorie_05_DE.md`, section 5.

© 2026 iAlexMG. Tous droits réservés.